

The Complexity Class NP

1 Definition of NP

Definition: NP

The complexity class **NP** is the class of languages whose **Yes** instances can be verified by a polynomial-time algorithm using a polynomial-size certificate.

More formally, a language L belongs to **NP** if and only if there exist:

- a two-input polynomial-time algorithm A ,
- a constant c ,
- and a certificate y with $|y| = \mathcal{O}(|x|^c)$,

such that:

$$L = \{x \in \{0,1\}^* : \text{there exists a certificate } y \text{ with } |y| = \mathcal{O}(|x|^c) \text{ such that } A(x, y) = 1\}$$

Verification View

Algorithm A verifies language L in polynomial time if:

$$x \in L \iff \exists y \text{ such that } A(x, y) = 1$$

The certificate y must be short enough to write down in polynomial length.

2 Solving vs Verifying

Intuition

P contains problems that can be solved quickly.

NP contains problems for which a proposed solution can be verified quickly.

Example: Hamiltonian Cycle

For HAM-CYCLE, a certificate is a list of vertices in the order they appear in the cycle. The verifier checks:

1. The list contains every vertex exactly once.
2. Every consecutive pair has an edge.
3. The last vertex connects back to the first.

This can be done in polynomial time, so:

$$\text{HAM-CYCLE} \in \mathbf{NP}$$

3 Why P is Inside NP

Relationship Between P and NP

If a language can be decided in polynomial time, then it can also be verified in polynomial time.

Suppose:

$$L \in \mathbf{P}$$

Then there exists a polynomial-time algorithm that decides whether $x \in L$.

We can convert it into a verifier $A(x, y)$ that simply ignores the certificate y :

$$A(x, y) = 1 \quad \text{if and only if } x \in L$$

Conclusion

$$\mathbf{P} \subseteq \mathbf{NP}$$

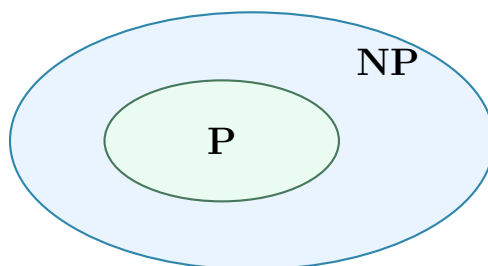


Figure: Every language in **P** is also in **NP**.

4 The P vs NP Question

Open Problem

It is unknown whether:

$$\mathbf{P} = \mathbf{NP}$$

or:

$$\mathbf{P} \neq \mathbf{NP}$$

Most researchers believe that:

$$\mathbf{P} \neq \mathbf{NP}$$

The intuition is that finding a solution from scratch may be much harder than checking a clearly presented solution.

Important Idea

Showing that a problem is in **NP** does not mean we know how to solve it quickly. It only means that if someone gives us a proposed solution, we can check it quickly.

5 Complements and co-NP

Complement of a Language

For a language L , its complement is:

$$\bar{L} = \{x \in \{0,1\}^* : x \notin L\}$$

The class **co-NP** is defined as:

$$\mathbf{co-NP} = \{L : \bar{L} \in \mathbf{NP}\}$$

Question

Is **NP** closed under complement?

That is, does:

$$L \in \mathbf{NP} \implies \bar{L} \in \mathbf{NP}$$

always hold?

This question is still unresolved.

Equivalently, no one knows whether:

$$\mathbf{NP} = \mathbf{co-NP}$$

6 P and co-NP

P is Closed Under Complement

If $L \in \mathbf{P}$, then $\bar{L} \in \mathbf{P}$.

Reason: a polynomial-time decider for L can be turned into a decider for $\bar{L}$ by swapping accept and reject.

Since:

$$\mathbf{P} \subseteq \mathbf{NP}$$

and:

$$\mathbf{P} \subseteq \mathbf{co-NP}$$

we get:

$$\mathbf{P} \subseteq \mathbf{NP} \cap \mathbf{co-NP}$$

7 Possible Relationships

Four Possible Scenarios

Researchers do not know the exact relationship among **P**, **NP**, and **co-NP**.

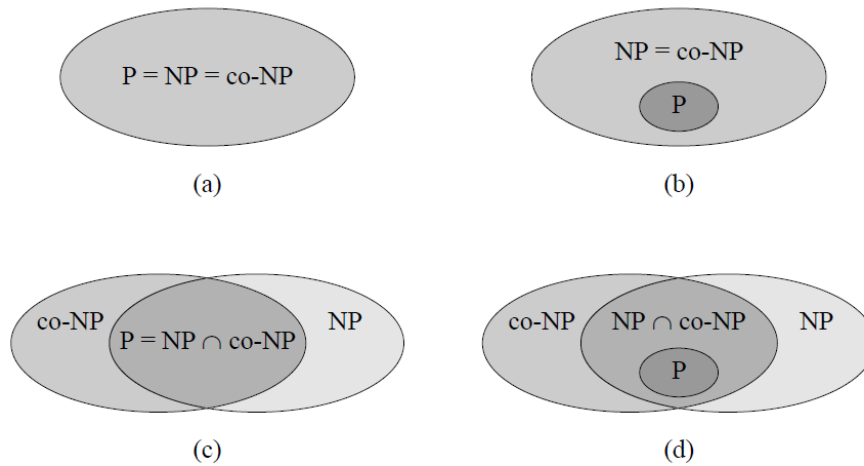


Figure 1: Four possibilities for relationships among complexity classes. In each diagram, one region enclosing another indicates a proper-subset relation. (a) $P = NP = co-NP$. Most researchers regard this possibility as the most unlikely. (b) If NP is closed under complement, then $NP = co-NP$, but it need not be the case that $P = NP$. (c) $P = NP$ and $co-NP$, but NP is not closed under complement. (d) NP not equal $co-NP$ and P not equal NP and $co-NP$. Most researchers regard this possibility as the most likely.

Most Likely Belief

Most researchers believe that:

$$P \neq NP$$

and:

$$NP \neq co-NP$$

So scenario (d) is often regarded as the most likely.

8 Why NP-Completeness Matters

NP-Complete Problems

NP-complete problems are the hardest problems in **NP** in the sense that every problem in **NP** can be reduced to them in polynomial time.

Even if we cannot prove that a particular problem is impossible to solve quickly, proving it is **NP-complete** gives valuable evidence that the problem is computationally difficult.

Key Consequence

If any **NP-complete** problem has a polynomial-time algorithm, then:

$$P = NP$$

So finding a polynomial-time algorithm for one **NP-complete** problem would solve every problem in **NP** in polynomial time.

9 Key Ideas

Term	Meaning
P	Languages decidable in polynomial time.
NP	Languages verifiable in polynomial time using polynomial-size certificates.
Certificate	Extra information that helps verify that $x \in L$.
Verifier	A polynomial-time algorithm $A(x, y)$ that checks a certificate.
co-NP	Languages whose complements are in NP .
NP-complete	The hardest problems in NP under polynomial-time reductions.